Gebze Technical University

Computer Engineering Department

CSE 396 Computer Engineering Project

Module Documentation Report

MOD-05: Unity Digital Twin Module

Autonomous First Responder Fire & Rescue Robot

Name	Student ID	Role
Dicle Çoban	220104004088	Primary - UI, Map Visualization & Mocking
Nuri Ziya Kırtepe	210104004027	Secondary - WebSocket, JSON & Mocking
Evrim Doğa Solmaz	230104004042	Secondary - Audio Capture Pipeline

Instructor: Salih Sarp
Date: April 19, 2026
Version: 1.1

Contents

1 Executive Summary

Module 5 serves as the primary operator interface for the Rescue Robot system. Built using the Unity Engine, it provides a real-time "Digital Twin" visualization of the robot's state, environment, and mission progress. The module aggregates data from various sensors (vision, acoustics, environment) and presents them through a 2D/3D hybrid dashboard, allowing the operator to monitor telemetry and issue commands via Push-to-Talk (PTT).

2 System Architecture & Connections

2.1 Inter-Module Integration

Module 5 acts as the central data sink for the entire system. It maintains the following inter-module dependencies:

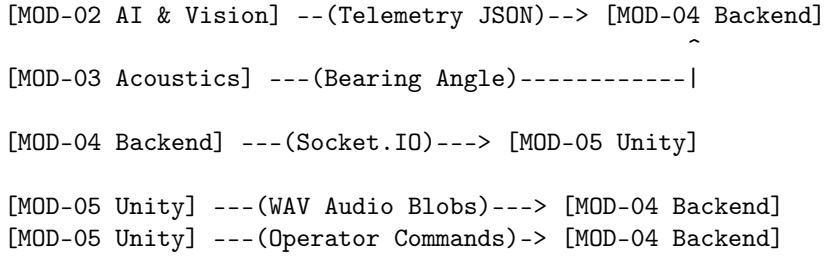

Figure 1: Module 5 Inter-Module Data Flow

2.2 Internal Data Flow

The module utilizes an event-driven architecture to ensure high performance and low coupling between components, as illustrated in the sequence below:

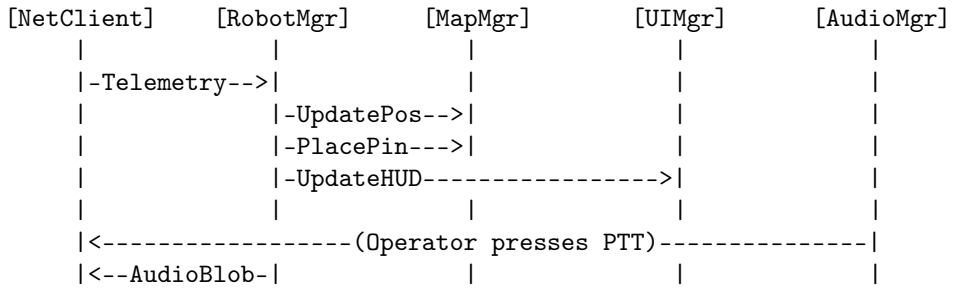

Figure 2: Module 5 Internal Sequence Diagram

3 Script Deep Dive & Design Choices

3.1 Core Architecture: `RobotManager.cs`

- **Role:** The centralized coordinator and lifecycle manager.
- **Design Choice:** A "Singleton-lite" coordinator pattern is employed. Instead of every UI element subscribing to the network, only the `RobotManager` listens. It then distributes data to the specialized managers (Map, UI, Acoustic).

- **Key Implementation:** Manages the instantiation of the `INetworkClient` (swapping between Real and Mock) and automatically handles interference by disabling legacy test scripts (`MockTelemetryTester`) when the new file-based mock mode is active.

3.2 Networking Abstraction: `INetworkClient.cs`

- **Role:** Interface defining the communication contract.
- **Design Choice:** Utilizing an interface successfully decoupled the high-level dashboard logic from the low-level socket logic, enabling the creation of `FileNetworkClient` without altering visualization scripts.

3.3 Real-Time Communication: `WebSocketClient.cs`

- **Role:** Handles the Socket.IO / Engine.IO protocol.
- **Implementation:** Uses `ClientWebSocket` for asynchronous, non-blocking I/O. It features a custom JSON parser that handles field name variations (e.g., `posX` vs `pos_x`) to ensure robustness against backend changes. `SynchronizationContext` is used to ensure network events are safely marshaled back to the Unity Main Thread for UI updates.

3.4 Mock Testing Infrastructure: `FileNetworkClient.cs`

- **Role:** Simulates a live backend using local JSON files.
- **Design Choice:** A "Packet Streaming" loop was implemented. It reads a list of telemetry objects from `StreamingAssets` and fires events at a controlled interval, effectively mimicking the behavior of the real robot for demonstration purposes.

3.5 Visualization Layer: `MapManager.cs`

- **Role:** 2D Tactical Map and Pin Management.
- **Design Choice:** Utilizes a coordinate mapping system (`GridToWorldPosition`) to translate abstract robot coordinates into Unity's 3D space.
- **Key Feature:** Implements a `cellKey` dictionary to prevent "pin cluttering"—if multiple detections happen in the same grid cell, the old pin is seamlessly replaced by the latest status.

3.6 Human Interface: `UIManager.cs`

- **Role:** HUD (Heads-Up Display) management.
- **Implementation:** Provides dynamic text and color updates for temperature and smoke. It uses an `UpdateHUD` convenience method to refresh the entire dashboard in a single call, reducing processing overhead.

3.7 Audio Pipeline: `AudioManager.cs`

- **Role:** Push-to-Talk (PTT) capture and encoding.
- **Design Choice:** A manual WAV encoder (`EncodeToWav`) was implemented within the script.
- **Rationale:** Standard Unity `AudioClip` data is floating-point PCM. Most backend STT engines require 16-bit signed integer WAV files. The custom encoder handles this conversion in real-time, including RIFF header generation.

3.8 Acoustic Visualization: `MapManager_AcousticBeam.cs`

- **Role:** Visualizing the direction of acoustic distress calls.
- **Implementation:** Receives bearing angles from `RobotManager` and dynamically rotates and scales an "Acoustic Beam" arrow or cone in the 3D world to guide the operator toward the source of the sound.

3.9 Legacy Demo Driver: `MockTelemetryTester.cs`

- **Role:** Hardcoded route simulation.
- **Context:** Used prior to the development of the File-Based Mock system. It contains a static array of route points and forces the robot to follow them.
- **Integration:** Currently, `RobotManager` detects if this script is active and automatically disables it to prevent data conflicts with the JSON-based system.

3.10 Simulated Environment: `VirtualSensor.cs` & `VictimInfo.cs`

- **Role:** Unity-side hardware simulation.
- **Functionality:** `VirtualSensor` simulates a YOLO camera and Ultrasonic sensor using `Physics.OverlapSphere` to detect `Victim` objects. `VictimInfo` serves as a metadata container attached to 3D victim models, storing severity status.
- **Design Choice:** This configuration allows testing of the "Detection $\rightarrow$ Pin Placement" flow entirely within Unity, bypassing the need for an active physical Raspberry Pi camera.

4 Data Contracts (`DataContracts.cs`)

Standardization is critical to module interoperability. The `TelemetryData` struct serves as the "common language" of the project:

Field	Type	Description
`posX / posY`	`float`	2D Grid coordinates.
`temperature`	`float`	Environmental sensor data.
`smokeDetected`	`bool`	Fire safety indicator.
`victimStatus`	`enum`	AI classification result (Standing / Lying / Trapped).
`acousticAngle`	`float`	Direction of distress call.

Table 1: Core Telemetry Structure

5 Demo Mode & Presentation Readiness

The module features a dedicated **Presentation Mode** accessible via the Unity Inspector to facilitate seamless demonstrations:

1. **Toggle:** `Use Mock File Data` flag is enabled.
2. **Logic:** Disables live network listeners and the legacy `MockTelemetryTester`.
3. **Payload:** Automatically loads the `mock_telemetry.json` file.
4. **Result:** A fully automated "playback" of a rescue mission is executed, which is ideal for office presentations where a real robot or backend server cannot be physically deployed.

6 Conclusion

Module 5 represents a robust, extensible visualization platform. Its decoupled architecture, primarily leveraging interface-based networking, alongside its integrated demonstration capabilities, makes it a highly reliable tool for both live mission operations and academic presentations.